

CMR SPECIALIST

Implementation & Hosting Manual

Lean case-centered agent (spec v2.0): what was built, how it runs, how to test it, and how to host it — from laptop to cloud.

Item	Detail
Scope	Stages 0–3: foundation, case memory, knowledge retrieval, task agent + operator UI
Stack	FastAPI + SQLAlchemy + SQLite (local) / PostgreSQL (cloud) + background worker
Entry points	Web: harness.api.main · Worker: harness.worker.worker
Tests	tests/test_lean_spec.py — 7 checks, all passing

NOTE. Start with one real task type (payment_reconciliation) and prove it end to end before adding more tools, channels, or agents.

1 · What was implemented

The codebase was rebuilt around one principle from the spec: **the case database is the memory**. Every input, extracted fact, run, tool call, citation, approval, and state change is a database row. The model proposes; typed schemas, policy checks, and deterministic code enforce.

Module map

Module	Responsibility
configs/settings.py	Typed config (DATABASE_URL, API key, ports, worker/retrieval limits)
harness/store/ (database, models, repository)	11-table durable memory, versioned extraction, job leasing
harness/agent/ (guard, intent, extractor, retriever, planner, policy, validator, service)	Controlled harness: 5-way intent router, typed extraction, active-only retrieval, bounded plans
harness/tools/registry.py	6 allowlisted tools with timeout, retry, permission level
harness/worker/worker.py	Durable worker loop: lease, execute, retry, resume after restart
harness/api/main.py + schemas.py	Spec interfaces: cases, messages, tasks, approvals, knowledge, health
frontend/index.html	Operator console: submit work, follow-ups, tasks, approvals, knowledge upload
docker/	Web + worker containers; optional local Postgres service
tests/test_lean_spec.py	Exit checks for Stages 0–3

Table 1 — where each spec concern lives in code.

Data model (all rows carry case/task/run linkage for audit)

Table	Key content
cases	id, title, status, source channel, owner, timestamps
case_inputs	case_id, raw text (verbatim), sender, received time
extracted_fields	name, type, raw + normalized value, confidence, validation, version, accepted_by
tasks	case_id, task_type, status, plan, result, idempotency key, attempts, lease info
agent_runs / run_steps	intent, model, timings; per-step action, tool, args, result, retries
knowledge_documents / versions / chunks	file metadata + hash; version status; chunk text + locator
citations	run → document version → chunk + quoted span + score
approvals	task, requested action, requester/approver, decision, reason, times
case_events	append-only actor / event / state transition / payload history

How the seven spec rules are enforced

Rule	Enforcement
Case memory first	Nothing lives only in chat context; service.py persists input before any reasoning
Extracted data is evidence	New revision per correction (version+1); raw, normalized, source, run kept
Retrieval before generation	retriever.py queries active versions only; empty result → bounded fallback
Harness controls model	Pydantic schemas in agent/schemas.py accept or reject every proposal
Tools allowlisted	REGISTRY dict; unknown tool → not_allowlisted error, never executed
Durable + idempotent jobs	DB-backed lease; per-step idempotency keys skip already-done steps
Scale after measurement	/health exposes oldest-queued-job age; scale only when it persists

2 · How it works

Knowledge question

1. **Store input** — POST /cases persists the case and verbatim message first. 2. **Classify** — intent router returns KNOWLEDGE_QUERY + confidence. 3. **Extract** — entities (NIN, phone, RRR, plate...) stored as versioned fields. 4. **Retrieve** — active knowledge versions only; chunk IDs recorded. 5. **Generate + validate** — answer assembled from spans; citations required. Weak retrieval or failed validation returns the bounded no-evidence fallback instead of guessing.

Automation task (one real type first: payment_reconciliation)

1. **Store task** — POST /cases/{id}/tasks with idempotency key (reposts return the same task). 2. **Validate fields** — required checklist is remita_rrr + payment_date + account_identifier; missing values park the task in **waiting_for_input** with per-field prompts. 3. **Plan + policy** — plan stored before execution (validate_fields → knowledge_lookup → payment_status_check → draft_solution); sensitive tools pause in **waiting_approval**. 4. **Worker executes** — each step stores arguments, result, status, retries. 5. **Follow-up resumes** — new message re-runs extraction and flips waiting tasks back to queued.

WARNING. The system never claims an external event occurred (payment confirmed, account updated, ownership transferred) unless a verified tool result or human approval proves it.

3 · Run it locally (Stage 0)

Prerequisites

- **Python 3.10+** (3.11/3.12 verified) and **pip**.
- Core deps install with plain pip: **fastapi**, **uvicorn**, **pydantic**, **sqlalchemy**, **pypdf**, **requests**, **pyyaml** (heavy ML packages are optional and NOT required).
- Test client deps: **pytest**, **httpx**. Postgres driver **psycopg[binary]** only when using Postgres.
- Ports: web serves on **8080** by default (HARNESS_PORT). No other services needed for SQLite mode.

Install

```
cd /Users/mac/Desktop/cmr_specialist
pip install -r requirements.txt # lean core (no torch/GPU needed)
pip install pytest httpx pypdf # tests + PDF knowledge ingestion
```

Configure (all optional — defaults run out of the box)

Variable	Default	Purpose
DATABASE_URL	sqlite:///.../data/cmr_cases.db	System of record; point at Postgres for cloud
HARNESS_API_KEY	cmr-secret-key-2026	Send as X-API-Key header (empty disables check in dev)
HARNESS_PORT / HOST	8080 / 0.0.0.0	Web bind address
HARNESS_DATA_DIR	./data	Local file area
INFERENCE_PROVIDER	mock	mock http (OpenAI-compatible endpoint)
WORKER_POLL_INTERVAL_S	2.0	Queue poll cadence

Start the two processes (two terminals, one codebase)

```
# Terminal 1 – web process
uvicorn harness.api.main:app --host 0.0.0.0 --port 8080

# Terminal 2 – worker process (same image, same DB)
python -m harness.worker.worker

# Health: expect database: connected
curl localhost:8080/health
```

Seed knowledge and ask a grounded question

```
# Upload a procedure text (PDF also accepted)
curl -X POST localhost:8080/knowledge/documents \
-F "file=@renewal_steps.txt;type=text/plain"

# → {"document_id": ..., "version_id": "<VER>", "chunks": N}

# Publish the version so retrieval can use it
curl -X POST localhost:8080/knowledge/versions/<VER>/publish \
-H 'Content-Type: application/json' -d '{}

# Ask – answer carries citations, or a clear fallback if no evidence
curl -X POST localhost:8080/cases \
-H 'Content-Type: application/json' -d '{"text":"How do I renew my CMR certificate?"}'
```

Run a task end to end (payment reconciliation)

```
# 1. Submit (parks in waiting_for_input, tells you what is missing)
curl -X POST localhost:8080/cases -H 'Content-Type: application/json' \
```

```
-d '{"text":"Reconcile my payment for account john@example.com"}'
# → {"task_id": "<TASK>", "status": "waiting_for_input", "missing": ["remita_rrr", ...]}

# 2. Supply the missing facts on the same case
curl -X POST localhost:8080/cases/<CASE>/messages -H 'Content-Type: application/json' \
-d '{"text":"My RRR is 123456789012, paid yesterday for account john@example.com"}'

# 3. Worker picks it up (or force one pass) and polls to completed
curl localhost:8080/tasks/<TASK> # steps, tool results, citations
```

NOTE. Re-posting a task with the same idempotency_key returns the existing task — safe to retry clients and safe against double execution.

4 · Testing (exit checks)

Automated suite

```
python -m pytest tests/ -q
# expected: 7 passed, 1 skipped
# (tests/test_harness.py is a skipped pre-v2 prototype; the spec suite is test_lean_spec.py)
```

Test	Stage	What it proves
test_stage0_health	0	/health ok, version 2.0.0, database connected
test_stage1_case_memory_versioned	1	Inputs preserved; phone correction → version 2, source + run kept
test_stage2_knowledge_cited_or_fallb ack	2	Published chunks → cited answer; unrelated query → fallback
test_stage3_task_async_idempotent_an d_waiting	3	Missing RRR → waiting_for_input; worker completes; same key = same task
test_stage3_approval_gate	3	approval:always → waiting_approval; approve → completes without gate-loop
test_intent_enum_and_unsupported	3	5-way router incl. UNSUPPORTED + NEEDS_CLARIFICATION
test_extractor_task_schema	1–3	Required-field checklist per task type incl. all-missing case

Manual checklist before calling a stage done

- Restart web + worker mid-task: the job is re-leased and continues from the last committed step.
- Post the same task twice with one idempotency key: one task row, no duplicate side effects.
- Ask with an empty knowledge base: bounded fallback, no invented procedure text.
- Request something out of policy (e.g. bypass verification): UNSUPPORTED refusal.
- Approve then reject paths: approved completes, rejected fails with the recorded reason.
- GET /cases/{id} shows messages, field revisions, tasks, and outputs without DB access.

5 · Hosting

Shape: one small web instance, one worker instance, one managed database. Keep it provider-neutral until budget and region are confirmed.

Option A — single VPS with Docker Compose (simplest, staging-ready)

```
cd docker
# default runs on SQLite inside a volume – zero external services
docker compose up --build
# web → http://<host>:8080 (health: /health, console: /)
```

```
#
# Staging with local Postgres: uncomment the db service in
# docker/docker-compose.yml, then:
DATABASE_URL=postgresql+psycopg://cmr:cmr-secret@db:5432/cmr docker compose up --build
```

Option B — managed platform (Render / Railway / Fly / VPS + managed Postgres)

- Create a **managed PostgreSQL** database (Neon, Supabase, RDS, or your platform's Postgres).
- Deploy **two services from the same image/commit**: web (uvicorn harness.api.main:app --host 0.0.0.0 --port 8080) and worker (python -m harness.worker.worker).
- Set **DATABASE_URL=postgresql+psycopg://user:pass@host:5432/cmr** plus HARNESS_API_KEY on both services; install **psycopg[binary]** in the image for Postgres.
- Point a health check at **GET /health**; alert when **oldest_queued_job_age_s** exceeds your target.
- Start with 1 web + 1 worker (concurrency 1–4). Scale only when queue age, provider limits, CPU, or DB latency persist.

Production env (set on both web and worker)

Variable	Example	Notes
DATABASE_URL	postgresql+psycopg://...@...:5432/cmr	Managed Postgres; automated backups on
HARNESS_API_KEY	(long random secret)	Staff/clients send as X-API-Key
INFERENCE_PROVIDER	mock (or http)	http needs INFERENCE_HTTP_URL + key
WORKER_POLL_INTERVAL_S	2.0	Lower only if queue age demands it

NOTE. First deploy proves one workflow end to end: upload + publish knowledge, ask a cited question, run one payment_reconciliation task, approve it, then snapshot/restore the database and confirm history and queued work survive.

6 · Operations & troubleshooting

Symptom	Check	Fix
/health shows database: error	DATABASE_URL, DB reachable, credentials	Fix URL; restart both processes
Tasks stuck in queued	oldest_queued_job_age_s rising; worker logs	Start/restart worker; raise concurrency to 2–4
Task in waiting_for_input	GET /tasks/{id} → waiting_reason.missing	POST the missing facts to /cases/{id}/messages
Task in waiting_approval	GET /tasks/{id} → approvals	POST /tasks/{id}/approvals approved/rejected
Cited answer never appears	version status must be active	POST /knowledge/versions/{id}/publish
PDF upload rejected	pypdf installed; text extractable	pip install pypdf; use text PDF, not scanned images
Duplicate tasks	idempotency_key on create	Reuse keys; reposts return the same task id

Security & privacy baseline (launch hardening, Stage 5)

- Secrets via environment, never in code; least-privilege DB and tool credentials; staff auth at the edge.
- PII masking in logs; field-level access rules; retention categories; deliberate deletion workflow.
- File validation on upload; HTTPS termination; automated DB backups with a tested restore.
- Structured logs carry request/case/task/run IDs; track LLM latency, tokens, cost, failure category.

Appendix A · API reference

Method & path	Purpose
POST /cases	Create case from text (+ optional title/sender/owner); runs intake
POST /cases/{id}/messages	Follow-up: attach info, re-extract, resume waiting tasks
POST /cases/{id}/tasks	Queue automation task (task_type, instructions, approval_required)
GET /cases/{id}	Case state: messages, versioned fields, tasks, outputs, recent events
GET /tasks/{id}	Progress: status, waiting reason, plan, steps, approvals, result/error
POST /tasks/{id}/approvals	Approve or reject a guarded action {decision, approver, reason}
POST /knowledge/documents	Upload source (txt/md/pdf) → chunked version (ready)
POST /knowledge/versions/{id}/publish	Activate version for retrieval (archives siblings)
GET /knowledge/documents	List documents + version statuses
GET /health	Readiness, DB, oldest queued-job age, worker heartbeat

Appendix B · Task lifecycle

queued → **leased** → **running** → **completed**, with side exits to **waiting_for_input** (missing facts), **waiting_approval** (sensitive step), **failed** (terminal error + error_ref), and **dead_letter** (retries exhausted). Expired leases are reclaimed by the next worker.